

SENAI GUARULHOS

Curso de Python com Framework

GABRIELY SANTOS ARAÚJO

Sistema CRUD de Usuários com Flask e FastAPI

GUARULHOS

2026

SENAI GUARULHOS

GABRIELY SANTOS ARAÚJO

Sistema CRUD de Usuários com Flask e FastAPI

Trabalho apresentado ao Curso de Python com Framework
como requisito para avaliação da disciplina.

Professor(a): Beatriz

GUARULHOS

2026

SUMÁRIO

1. Introdução	1
2. Visão Geral do Projeto	2
3. Objetivos	3
4. Tecnologias Utilizadas	4
5. Requisitos Funcionais	5
6. Requisitos Não Funcionais	6
7. Arquitetura do Sistema	7
8. Estrutura do Projeto	8
9. Regras de Negócio	9
10. Processo de Desenvolvimento ...	10
11. Testes	11
12. Deploy	12
13. Conclusão	13

1. INTRODUÇÃO

A tecnologia está cada vez mais presente no desenvolvimento de soluções para gerenciamento de informações e automação de processos. Nesse contexto, sistemas CRUD são amplamente utilizados para realizar operações de cadastro, consulta, atualização e exclusão de dados.

Este trabalho apresenta a documentação de um sistema CRUD de usuários utilizando Flask e FastAPI, com foco na organização do projeto, arquitetura de software, autenticação de usuários e utilização de banco de dados PostgreSQL.

O projeto foi desenvolvido com o objetivo de aplicar conceitos modernos de desenvolvimento backend, APIs RESTful, segurança com JWT e boas práticas utilizadas no mercado de tecnologia.

2. VISÃO GERAL DO PROJETO

O presente projeto consiste no desenvolvimento de um sistema CRUD de usuários utilizando Flask e FastAPI. O sistema será responsável pelo gerenciamento de usuários, permitindo operações de cadastro, login, consulta, atualização e exclusão de registros.

A aplicação utilizará PostgreSQL como banco de dados e SQLAlchemy como ORM para manipulação das informações. Além disso, serão aplicados conceitos de autenticação JWT, organização em camadas e desenvolvimento de APIs RESTful.

O projeto foi planejado seguindo boas práticas utilizadas no mercado de desenvolvimento de software, com foco em segurança, organização, escalabilidade e manutenção do sistema.

3. OBJETIVOS

O objetivo deste projeto é desenvolver a documentação de um sistema CRUD de usuários utilizando Flask e FastAPI, aplicando conceitos modernos de desenvolvimento backend e organização de software.

Além disso, o projeto busca utilizar tecnologias amplamente adotadas no mercado, como PostgreSQL, SQLAlchemy e autenticação JWT, visando segurança, desempenho e escalabilidade da aplicação.

O sistema também tem como objetivo proporcionar melhor entendimento sobre APIs RESTful, estruturação de projetos em camadas e boas práticas de desenvolvimento de software.

4. TECNOLOGIAS UTILIZADAS

Para o desenvolvimento do sistema serão utilizadas tecnologias modernas amplamente utilizadas no mercado de desenvolvimento web e backend. Essas ferramentas possuem como objetivo garantir organização, desempenho, segurança e facilidade de manutenção da aplicação.

TECNOLOGIA	FUNÇÃO
FLASK	DESENVOLVIMENTO BACKEND DA WEB
FASTAPI	CRIAÇÃO DE APIS REST
POSTGRESQL	BANCO DE DADOS
SQLALCHEMY	ORM PARA BANCO DE DADOS
JWT	AUTENTICAÇÃO DE USUÁRIOS
DOCKER	PADRONIZAÇÃO DO AMBIENTE
AWS	HOSPEDAGEM E DEPLOY

5. REQUISITOS FUNCIONAIS

Os requisitos funcionais representam as funcionalidades que o sistema deverá executar para atender às necessidades dos usuários e garantir o funcionamento correto da aplicação.

RF01 — Cadastro de usuários

RF02 — Login de usuários

RF03 — Operações CRUD

RF04 — Busca e paginação de registros

RF05 — Controle de permissões

RF06 — API REST

RF07 — Documentação Swagger/OpenAPI

RF08 — Logs do sistema

6. REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais representam características relacionadas ao desempenho, segurança, disponibilidade e qualidade do sistema, garantindo melhor funcionamento e experiência para os usuários.

RNF01 — Desempenho

RNF02 — Escalabilidade

RNF03 — Segurança

RNF04 — Disponibilidade

RNF05 — Compatibilidade

RNF06 — Manutenibilidade

RNF07 — Observabilidade

7. ARQUITETURA DO SISTEMA

A arquitetura do sistema foi planejada utilizando organização em camadas, visando melhor separação de responsabilidades, manutenção e escalabilidade da aplicação.

O sistema contará com Flask para gerenciamento da aplicação web e FastAPI para desenvolvimento das APIs RESTful. A camada de serviços será responsável pelas regras de negócio, enquanto os repositórios realizarão a comunicação com o banco de dados PostgreSQL.

Frontend → Flask → FastAPI → Services → Repositories → PostgreSQL

8. ESTRUTURA DO PROJETO

O projeto será organizado em diretórios específicos para melhor estruturação e manutenção do sistema. A divisão das pastas tem como objetivo separar responsabilidades e facilitar o desenvolvimento da aplicação.

```
project/
```

```
├─ app/
```

└─ tests/

└─ migrations/

└─ docker/

└─ main.py

A pasta “app” será responsável pelo código principal da aplicação. A pasta “tests” armazenará os testes do sistema, enquanto “migrations” será utilizada para controle de alterações do banco de dados. A pasta “docker” conterá arquivos de configuração do ambiente da aplicação.

9. REGRAS DE NEGÓCIO

As regras de negócio representam condições e restrições definidas para garantir o funcionamento correto e seguro do sistema.

- Emails devem ser únicos no sistema.
- Apenas administradores poderão excluir registros.
- O token JWT deverá possuir tempo de expiração.
- Os endpoints protegidos exigirão autenticação do usuário.

10. PROCESSO DE DESENVOLVIMENTO

O desenvolvimento do projeto seguirá etapas organizadas para garantir melhor planejamento, implementação e manutenção da aplicação.

Inicialmente será realizado o levantamento dos requisitos do sistema, identificando funcionalidades, regras de negócio e tecnologias necessárias para o desenvolvimento.

Em seguida, será feita a modelagem da arquitetura do sistema e organização das pastas do projeto. Após essa etapa, ocorrerá o desenvolvimento das funcionalidades utilizando Flask, FastAPI e PostgreSQL.

Posteriormente, serão realizados testes para validação das funcionalidades e segurança da aplicação. Por fim, o sistema poderá ser disponibilizado em ambiente de deploy utilizando serviços em nuvem.

11. TESTES

Os testes possuem como objetivo verificar o funcionamento correto da aplicação e garantir maior qualidade e segurança para o sistema.

Serão realizados testes unitários para validação de funções isoladas, testes de integração para comunicação entre componentes e testes de autenticação para verificar a segurança dos acessos ao sistema.

Os testes contribuirão para identificação de falhas, melhoria da estabilidade da aplicação e validação dos requisitos definidos no projeto.

12. DEPLOY

O deploy da aplicação poderá ser realizado em plataformas de hospedagem em nuvem, permitindo que o sistema fique disponível para acesso online.

Serviços como AWS, Render e Railway poderão ser utilizados para hospedagem da aplicação, banco de dados e gerenciamento da infraestrutura do sistema.

Além disso, ferramentas como Docker poderão auxiliar na padronização do ambiente da aplicação, facilitando a implantação e manutenção do projeto.

13. CONCLUSÃO

O presente trabalho apresentou a documentação de um sistema CRUD de usuários utilizando Flask e FastAPI, abordando conceitos relacionados ao desenvolvimento backend, APIs RESTful, autenticação JWT e organização de projetos de software.

Durante o desenvolvimento da documentação foram definidos requisitos funcionais, requisitos não funcionais, arquitetura do sistema, regras de negócio e etapas do processo de desenvolvimento da aplicação.

O projeto permitiu melhor compreensão sobre tecnologias modernas utilizadas no mercado de desenvolvimento web, além de demonstrar a importância da organização, planejamento e boas práticas na construção de sistemas.

REFERÊNCIAS

FLASK. Documentation. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 07 maio 2026.

FASTAPI. Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 07 maio 2026.

POSTGRESQL. Documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 07 maio 2026.

SQLALCHEMY. Documentation. Disponível em: <https://www.sqlalchemy.org/>. Acesso em: 07 maio 2026.

